

## Transmitting And Receiving Packets Using STAR-API

STAR-Dundee's new software stack, STAR-System, provides a consistent programming interface for accessing STAR-Dundee router and interface devices, including the STAR-Dundee SpaceWire PCI Mk2, cPCI Mk2 and PCIe devices and the new STAR-Dundee SpaceWire Brick Mk2 and Router Mk2S devices. The main API provided with STAR-System, STAR-API, provides functions for getting the list of devices connected to a PC, opening channels to access those devices, and then transmitting and receiving traffic over those channels.

This application note describes some of the functions of STAR-API which can be used to transmit and receive packets, and includes some example code to demonstrate opening a channel to a device, transmitting a packet on that channel, receiving a packet on the channel, and then closing the channel. The examples are written in C, but the API can just as easily be used in C++.

A separate application note, "**Implementing A Simple RMAP Initiator Using The STAR-System APIs**", expands upon this application note to demonstrate how the code presented here can be combined with the RMAP Packet Library to implement a simple RMAP initiator. Further information on the APIs and the functions they provide can be found in the STAR-System documentation.

### SpaceWire Devices

The list of SpaceWire devices connected to the current PC can be obtained by calling `STAR_getDeviceList()`. This returns an array of `STAR_DEVICE_ID`s, which should be destroyed by calling `STAR_destroyDeviceList()` when it is no longer required. The number of elements in the device array is obtained by passing a pointer to a `U32` variable to the function. `U32`s are used in STAR-API to represent 32-bit unsigned integers. There are corresponding types provided for 8-bit and 16-bit unsigned integers (`U8` and `U16`). The variable pointed to by the `U32` pointer passed to `STAR_getDeviceList()` will be updated to contain the length of the list:

```
STAR_DEVICE_ID *pDeviceList;
U32 deviceCount, i;

pDeviceList = STAR_getDeviceList(&deviceCount);

for (i = 0; i < deviceCount; i++)
{
    /** \todo Do something with the device */
}

STAR_destroyDeviceList(pDeviceList);
```

The device identifiers returned in the device list can be passed to a number of functions to get the properties of a device. For example, the type of the device as a string, e.g. "SpaceWire PCI Mk2", can be obtained by calling `STAR_getDeviceTypeAsString()`. The string returned should then be destroyed by calling `STAR_destroyString()` when it is no longer required:

```

char *deviceType;
STAR_DEVICE_ID deviceId = pDeviceList[0];

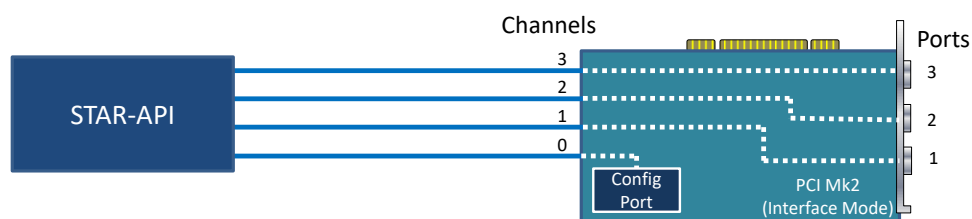
deviceType = STAR_getDeviceTypeAsString(deviceId);
if (deviceType)
{
    (void)printf("Device type = %s.\n", deviceType);
    STAR_destroyString(deviceType);
}
else
{
    (void)puts("Unable to obtain the device type.");
}

```

## Device Channels

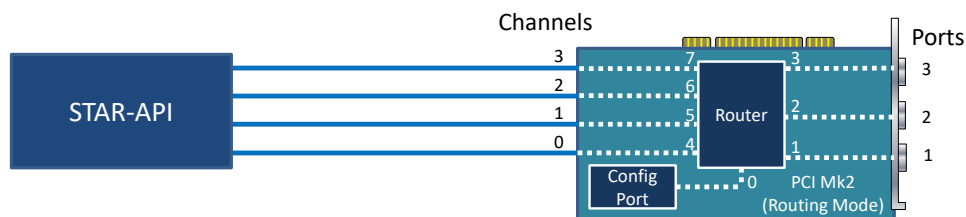
In STAR-API, all packets to be transmitted to a device and all packets received from a device are transferred between the device and the API over a previously opened channel. Each device has a certain number of channels which can be opened. On an interface device, or a device operating in interface mode, these channels are likely to attach directly to the links of the device. On a routing device, or a device operating in routing mode, these channels will attach to the external ports of the device's router.

The channels provided by a device can be obtained by calling `STAR_getDeviceChannels()`, which returns a bit mask indicating which of the channels exist for this device. The PCI Mk2, cPCI Mk2, PMC Mk2 and PCIe devices provide channels 0, 1, 2 and 3. When in interface mode, the default mode for these devices, channels 1, 2 and 3 correspond to links 1, 2 and 3, and any packets transmitted over these channels will be transmitted over the corresponding link. In this mode, channel 0 provides a channel for accessing the configuration port of the device, which is used by the Device Configuration APIs to configure the device. It is strongly recommended that channel 0 is not used for any other purpose, as traffic to this port will be passed directly to the configuration port. The figure below describes the relationship between channels and ports in interface mode on these devices.



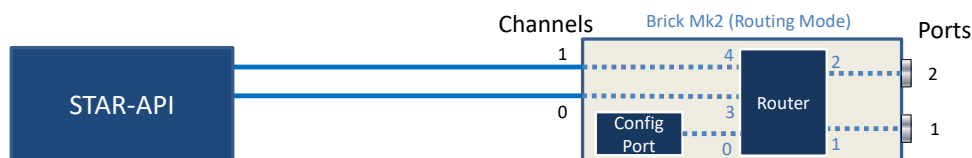
**Relationship between channels and ports on a PCI device in interface mode**

When these devices are in routing mode, channels 0, 1, 2 and 3 correspond to the four external ports on the device's router, ports 4, 5, 6 and 7. Channel 0 will still be used by the Device Configuration APIs to configure the device, so it's recommended that external port 4 is not used for any other purpose if the Configuration APIs are to be used. The following figure shows how traffic on each of the channels is routed to the ports when these devices are in routing mode.



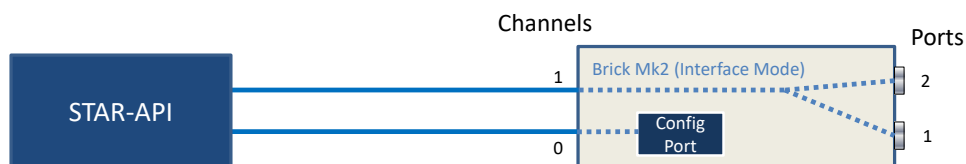
**Relationship between channels and ports on a PCI device in routing mode**

The configuration of channels is a little different with the Brick Mk2 and Router Mk2S. Both devices have only two channels, channel 0 and channel 1. As with the other devices, channel 0 will be used by the Device Configuration APIs, leaving channel 1 for transmitting and receiving packets. This means that when in interface mode (the default for the Brick Mk2), packets to/from all links must cross the same channel. As a result, when in interface mode, these devices require transmitted packets to include an address byte (e.g. the port number to route the packet out of) at the front of the packet. When transmitting packets, the device is effectively operating as a router. The figure below shows the relationship between channels and ports when a Brick Mk2 is in routing mode, and also when transmitting packets in interface mode.



**Relationship between channels and ports on a Brick Mk2 in routing mode**

Packets received on all ports are routed up the same channel when in the default interface mode, which means the device does not operate as a true interface device in this mode, as traffic from one port can block traffic from another port. The figure below describes the relationship between channels and ports in interface mode on the Brick Mk2.

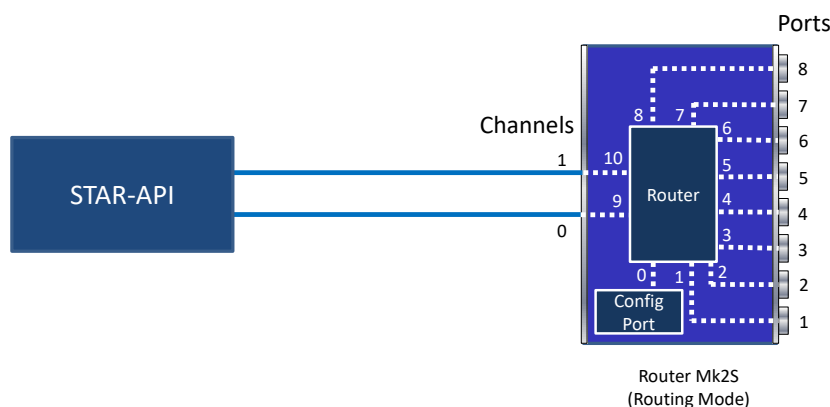


**Relationship between channels and ports on a Brick Mk2 in interface mode**

Although this would suggest that the Brick Mk2 cannot be used as a true interface device, all of the STAR-System interface and router devices support a mechanism called port routing. Port routing can be configured using the Device Configuration APIs, or using the Device Configuration application. Port routing allows a device to be configured so that all packets received on a port are automatically routed to another port. As a result, you can configure a Brick Mk2 so that packets received on port 1 are routed to port 3 (channel 0), and packets received on port 2 are routed to port 4 (channel 1). Note that if you're receiving packets on channel 0, you will not be able to use any of the Device Configuration API functions.

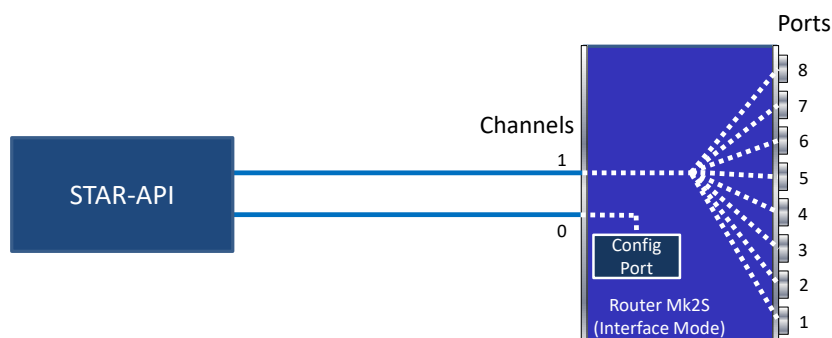
If you do not wish to add an address byte to the front of packets to be transmitted, you can also use port routing on the external ports. You could setup port routing so that packets received on port 4 (channel 1) are routed to port 2, and packets received on port 3 (channel 0) are routed to port 2. Note that if you do set this second port routing arrangement, you will not be able to access the configuration port, and as a result, use the Device Configuration APIs. This means you will not be able to change the port routing settings until the device is reset.

The Router Mk2S is very similar to the Brick Mk2, although it obviously has more ports. By default the device is in routing mode, as shown in the following figure. External ports 9 and 10 on the router correspond to channels 0 and 1.



**Relationship between channels and ports on a Router Mk2S in routing mode**

When in interface mode, packets from all ports are routed up channel 1, with the Device Configuration APIs using channel 0, as shown in the figure below. Again, port routing can be used to configure two ports to act as real interfaces, or to group multiple ports together over a single channel, e.g. ports 1 to 4 routed over channel 0, and ports 5 to 8 routed over channel 1.



**Relationship between channels and ports on a Router Mk2S in interface mode**

## Opening and Closing Device Channels

A channel to a local device can be opened by calling `STAR_openChannelToLocalDevice()`, passing in the channel number on the device to open. This function returns a `STAR_CHANNEL_ID` which can be used to identify the channel in future function calls, for example when transmitting and receiving packets. When a channel is no longer required, it should be closed by calling `STAR_closeChannel()`, passing in the channel ID returned when the channel was opened.

A device channel may be opened uni-directionally or bi-directionally. A channel on a device cannot be opened more than once in the same direction at the same time. For example, one application can open channel 1 of a device as an out channel to transmit packets, while another application can open channel 1 of the same device as an in channel to receive packets. The second application could not open that channel as an out channel, or an in/out channel, until the first application closes its channel. Channel 2 of the device could still be opened, however, in any direction. The direction of the channel is specified when it is opened by passing in one of the following values: `STAR_CHANNEL_DIRECTION_IN`, `STAR_CHANNEL_DIRECTION_OUT` or `STAR_CHANNEL_DIRECTION_INOUT`.

In addition to the device identifier and the channel number, a third parameter to the channel open function indicates whether traffic received on the channel should be buffered. If the channel is not

opened in buffered mode, packets will be discarded if they are received when no receive operation is waiting for packets. It's strongly recommended that the channel should be buffered for channels which are receiving traffic (in or in/out channels) as traffic may be lost. For channels which are transmitting only (out channels) the buffering status is ignored.

The following code shows the process of opening and closing a bidirectional channel on channel one of a device in buffered mode.

```
STAR_CHANNEL_ID channelId;
const unsigned char channelNumber = 1;

channelId = STAR_openChannelToLocalDevice(deviceId, STAR_CHANNEL_DIRECTION_INOUT,
    channelNumber, 1);
if (!channelId)
{
    (void)puts("Unable to open a channel to the device.");
    return EXIT_FAILURE;
}

/** \todo Do something with the channel */

(void)STAR_closeChannel(channelId);
```

## Transmitting Packets

Packets are transmitted in STAR-API by submitting transfer operations consisting of one or more stream items, representing the packets to be transmitted. A stream item can be a whole data packet, a part of a data packet (a data packet chunk) or a time-code. Other stream item types may be added to STAR-API in the future. To transmit a packet, you must first create a stream item to represent that packet. A packet stream item can be created by calling `STAR_createPacket()`. The parameters to this function include the address of the packet (which can be `NULL`), the bytes of the packet, the length of the packet, and the end of packet marker type, which can be EOP (`STAR_EOP_TYPE_EOP`), EEP (`STAR_EOP_TYPE_EEP`) or none (`STAR_EOP_TYPE_NONE`) to send a packet which is not terminated.

Once a stream item has been created, a transmit operation must be created to transmit that stream item. Transmit operations can transmit an array of stream items, allowing multiple packets to be transmitted in quick succession. A transmit operation is created by calling `STAR_createTxOperation()`, passing in the array of stream items and the number of elements in the array.

Once created, a transmit operation can be submitted to a previously opened channel using the function `STAR_submitTransferOperation()`, providing the channel identifier and the operation as parameters. Transmit and receive operations can be submitted using this function, and the two types of operation are collectively referred to as “transfer” operations. Once an operation has been submitted, its status can be obtained by calling `STAR_getTransferOperationStatus()`. To wait on the operation completing, `STAR_waitOnTransferOperationCompletion()` can be called. Both these functions take the transfer operation as a parameter, while `STAR_waitOnTransferOperationCompletion()` also takes a second parameter which is the time to wait on the operation completing in milliseconds. A value of `-1` can be used to wait indefinitely.

Once a transfer operation is complete, it can be disposed of by calling `STAR_disposeTransferOperation()`. This will stop the operation if it is in progress and free up all memory used. Any stream items which have been created must be destroyed separately by calling `STAR_destroyStreamItem()`. Alternatively, once a transfer operation has completed, it can be resubmitted by calling `STAR_submitTransferOperation()`, assuming it has not been disposed. This

can be useful when transmitting the same traffic multiple times or when repeatedly receiving traffic. In both cases, a transfer operation can be created once and the operation resubmitted each time it completes. Stream items can also be reused at any time. For example, a single data packet stream item could be created and provided in the calls to create to two transmit operations. These transmit operations could then be submitted to two different channels at the same time, resulting in the same packet being transmitted on each channel.

The code below shows the process of creating a packet and a corresponding transmit operation, submitting the operation, waiting on it completing, and then disposing of the operation and the packet.

```
STAR_STREAM_ITEM *pTxStreamItem;
STAR_TRANSFER_OPERATION *pTxTransOp;
STAR_SPACEWIRE_ADDRESS *pAddress = NULL;
unsigned char transmitBuffer[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
const int transmitBufferLength = sizeof(transmitBuffer);
STAR_TRANSFER_STATUS status;

/* Create a 10 byte packet to be transmitted, terminated with an EOP */
pTxStreamItem = STAR_createPacket(pAddress, transmitBuffer, transmitBufferLength,
    STAR_EOP_TYPE_EOP);
if (!pTxStreamItem)
{
    (void)puts("Couldn't create the stream item to be transmitted.");
    return EXIT_FAILURE;
}

/* Create a transfer operation to transmit the packet */
pTxTransOp = STAR_createTxOperation(&pTxStreamItem, 1);
if (!pTxTransOp)
{
    (void)puts("Couldn't create the transmit operation.");
    STAR_destroyStreamItem(pTxStreamItem);
    return EXIT_FAILURE;
}

/* Start transmitting the packet */
(void)STAR_submitTransferOperation(channelId, pTxTransOp);

/* Wait indefinitely on the transmit completing */
status = STAR_waitOnTransferOperationCompletion(pTxTransOp, -1);
if (status != STAR_TRANSFER_STATUS_COMPLETE)
{
    (void)printf("Could not transmit packet, error of %d.\n", status);
}

/* Dispose of the transmit operation */
(void)STAR_disposeTransferOperation(pTxTransOp);

/* Destroy the transmit packet */
STAR_destroyStreamItem(pTxStreamItem);
```

Since the above code is likely to be used quite often, STAR-API provides a wrapper function, `STAR_transmitPacket()`, to simplify these steps. This function isn't as flexible as using the functions from the previous example, because it only allows one packet to be sent at a time, and it may not provide the same performance as using the individual functions shown above. For example, a transmit operation can be resubmitted when it completes. It does greatly simplify the code required, however, as shown in the following example.



```
unsigned char transmitBuffer[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
const int transmitBufferLength = sizeof(transmitBuffer);
STAR_TRANSFER_STATUS status;

/* Transmit a packet, waiting indefinitely on being transmitted */
status = STAR_transmitPacket(channelId, transmitBuffer, transmitBufferLength,
    STAR_EOP_TYPE_EOP, -1);
if (status != STAR_TRANSFER_STATUS_COMPLETE)
{
    (void)printf("Could not transmit packet, error of %d.\n", status);
}
```

This example provides the same functionality as in the earlier transmit example, but with only one function call to STAR-API. The arguments to `STAR_transmitPacket()` include the channel to transmit on, the buffer to transmit, the length of the buffer, the end of packet marker type to add to the end of the buffer, and the timeout period in milliseconds. The timeout can be `-1` to wait indefinitely. The function returns the transfer status of the operation.

## Receiving Packets

Packets are received using steps similar to those required to transmit packets: an operation is created and it is then submitted to a channel. A receive operation is created by calling `STAR_createRxOperation()`, providing the number of stream items to receive and the types of stream item to receive. Passing `-1` for the number of stream items results in an unlimited number of stream items being received until the operation is cancelled. Note that the unlimited option should be used with caution as it can lead to out of memory errors.

The types of stream item that can be received are currently whole data packets, data packet chunks and time-codes. A receive operation can receive multiple stream item types, although note that only one of packets or packet chunks can be specified, not both. When receiving time-codes and data packet chunks, the sequence of time-codes and data characters will be maintained, e.g. a time-code may be received in the middle of a packet, which would result in a data packet chunk for the start of the packet, a time-code, and then a data packet chunk for the end of the packet being received. When receiving both time-codes and full packets, a time-code in the middle of a packet will be received by the operation before the packet. This is because the packet cannot be received by the operation until all of its bytes and end of packet marker have been received.

Submitting a receive operation, checking its status, waiting on it completing and disposing of it are all achieved using the same functions as for transmit operations (`STAR_submitTransferOperation()`, `STAR_getTransferOperationStatus()`, `STAR_waitOnTransferOperationCompletion()` and `STAR_disposeTransferOperation()`). When a receive operation completes, the stream items received can be obtained by calling `STAR_getTransferItem()` and `STAR_getTransferItemList()`. `STAR_getTransferItemCount()` returns the number of stream items received by the operation. Once the number of stream items has been obtained, the index of a stream item can be passed to `STAR_getTransferItem()` to obtain a pointer to that stream item. `STAR_getTransferItemList()` returns the full array of stream items, indicating the number of elements in that array by updating the variable pointed to by the pointer argument passed to the function. The stream items returned by these functions should not be destroyed as they are owned by the receive operation, and will continue to exist until the receive operation is disposed.

A stream item (`STAR_STREAM_ITEM`) is a structure with two properties: the stream item type (`itemType`) and the actual stream item (`item`). There are a number of functions for getting the properties of stream items, with different functions for each stream item type. When calling these

functions, the item property should be cast to the appropriate stream item type. For packets, the packet data can be obtained by calling `STAR_getPacketData()`, while the end of packet marker type can be obtained by calling `STAR_getPacketEOP()`. Note that a pointer to the data is returned by `STAR_getPacketData()`, and this must be destroyed when no longer required by calling `STAR_destroyPacketData()`.

The following example shows how to create a receive operation to receive one SpaceWire packet and then wait on it completing. Once the operation is complete, the bytes in the received packet are printed.

```
STAR_STREAM_ITEM *pRxStreamItem;
STAR_TRANSFER_OPERATION *pRxTransOp;
unsigned char *pReceiveBuffer;
STAR_TRANSFER_STATUS status;
unsigned int receiveBufferLength, i;

/* Create a transfer operation for receiving one packet */
pRxTransOp = STAR_createRxOperation(1, STAR_RECEIVE_PACKETS);
if (!pRxTransOp)
{
    (void)puts("Couldn't create receive operation, exiting.");
    return EXIT_FAILURE;
}

/* Start receiving a packet */
(void)STAR_submitTransferOperation(channelId, pRxTransOp);

/* Wait indefinitely on a packet being received */
status = STAR_waitOnTransferOperationCompletion(pRxTransOp, -1);
if (status != STAR_TRANSFER_STATUS_COMPLETE)
{
    (void)printf("Could not receive packet, error of %d.\n", status);
    (void)STAR_disposeTransferOperation(pRxTransOp);
    return EXIT_FAILURE;
}

/* Get the stream item received */
pRxStreamItem = STAR_getTransferItem(pRxTransOp, 0);
if (!pRxStreamItem)
{
    (void)puts("Unable to get the stream item received.");
    (void)STAR_disposeTransferOperation(pRxTransOp);
    return EXIT_FAILURE;
}

/* Get the bytes of the packet received */
pReceiveBuffer = STAR_getPacketData((STAR_SPACEWIRE_PACKET *)pRxStreamItem->item,
    &receiveBufferLength);
if (!pReceiveBuffer)
{
    (void)puts("Unable to get the contents of the packet.");
    (void)STAR_disposeTransferOperation(pRxTransOp);
    return EXIT_FAILURE;
}
```



```

/* Display the bytes in the packet */
(void)printf("Received packet contents:");
for (i = 0; i < receiveBufferLength; i++)
{
    (void)printf(" %2x", pReceiveBuffer[i]);
}
(void)puts(".");

/* Destroy the buffer containing the received packet */
STAR_destroyPacketData(pReceiveBuffer);

/* Dispose of the receive operation */
(void)STAR_disposeTransferOperation(pRxTransOp);

```

As with the transmit functions, STAR-API also provides a wrapper function, `STAR_receivePacket()`, to simplify the steps required to receive a single packet. This function can be used to receive a packet into a previously allocated buffer. As with `STAR_transmitPacket()`, it is not as flexible or as efficient as using the functions above. For example, the packet length is limited by the size of the buffer, and if the packet received is larger than the buffer, the rest of the packet is dropped. Again, to achieve the highest performance and lowest CPU utilisation, the individual functions described above should be used.

If you simply wish to receive one packet, however, and aren't concerned about performance, the example below shows code to receive a single packet, providing the same functionality as the example above.

```

unsigned char receiveBuffer[10];
STAR_TRANSFER_STATUS status;
unsigned int receiveBufferLength = sizeof(receiveBuffer), i;
STAR_EOP_TYPE eopType;

/* Receive a packet, waiting indefinitely on it being received */
status = STAR_receivePacket(channelId, receiveBuffer, &receiveBufferLength,
    &eopType, -1);
if (status == STAR_TRANSFER_STATUS_COMPLETE)
{
    /* Display the bytes in the packet */
    (void)printf("Received packet contents:");
    for (i = 0; i < receiveBufferLength; i++)
    {
        (void)printf(" %2x", receiveBuffer[i]);
    }
    (void)puts(".");
}
else
{
    (void)printf("Could not receive packet, error of %d.\n", status);
}

```

The arguments to `STAR_receivePacket()` include the channel to receive on, the buffer to receive in to, a pointer to a variable which should contain the length of the buffer and which will be updated to contain the length of the packet received, a pointer to a variable which will be updated to contain the end of packet marker type, and the timeout period in milliseconds. The timeout can be `-1` to wait indefinitely. The function returns the status of the operation.

## Complete Transmit and Receive Example Code

The code below brings all of the information provided above together to show how to transmit and receive a packet. It demonstrates the following functionality:

- Get the list of devices connected to the PC
- Get the type of the first device and the channels on that device
- Open a bidirectional channel to channel 1 of the device
- Create a transmit operation to transmit a single packet and a receive operation to receive a single packet
- Submit the transmit and receive operations on the channel
- Wait on the receive operation completing
- Display the contents of the received packet
- Dispose the transmit and receive operations and close the channel to the device

```
#include <stdio.h>
#include <stdlib.h>
#include <star-api.h>

int main(void)
{
    STAR_DEVICE_ID *pDeviceList, deviceId;
    U32 deviceCount, i;
    STAR_CHANNEL_MASK channelMask;
    STAR_CHANNEL_ID channelId;
    STAR_STREAM_ITEM *pTxStreamItem, *pRxStreamItem;
    STAR_TRANSFER_OPERATION *pRxTransOp, *pTxTransOp;
    STAR_SPACEWIRE_ADDRESS *pAddress = NULL;
    unsigned char transmitBuffer[] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
    unsigned char *pReceiveBuffer, *deviceType;
    const int transmitBufferLength = sizeof(transmitBuffer);
    STAR_TRANSFER_STATUS status;
    unsigned int packetCount, receiveBufferLength;
    const unsigned char channelNumber = 1;
    int returnStatus = EXIT_FAILURE;

    /* Get the list of devices */
    pDeviceList = STAR_getDeviceList(&deviceCount);

    /* Check there are devices present */
    if (!pDeviceList)
    {
        (void)puts("No devices present.");
        return EXIT_FAILURE;
    }

    (void)printf("%d device(s) present.\n", deviceCount);

    /* Store the device identifier of the first device */
    deviceId = pDeviceList[0];

    /* Destroy the device list */
    STAR_destroyDeviceList(pDeviceList);
```

```

/* Get the type of the device as a string */
deviceType = STAR_getDeviceTypeAsString(deviceId);
if (deviceType)
{
    (void)printf("Device type = %s.\n", deviceType);
    STAR_destroyString(deviceType);
}
else
{
    (void)puts("Unable to obtain the device type.");
}

/* Get the channels on the device */
channelMask = STAR_getDeviceChannels(deviceId);

/* Check that the channel exists on the device */
if (!(channelMask & (1 << channelNumber)))
{
    (void)printf("The device in use doesn't have a channel %d.", channelNumber);
    return EXIT_FAILURE;
}

/* Open a bidirectional buffered channel to the device */
channelId = STAR_openChannelToLocalDevice(deviceId, STAR_CHANNEL_DIRECTION_INOUT,
    channelNumber, 1);
if (!channelId)
{
    (void)puts("Unable to open a channel to the device.");
    return EXIT_FAILURE;
}

/* Create a packet to be transmitted, terminated with an EOP */
pTxStreamItem = STAR_createPacket(pAddress, transmitBuffer, transmitBufferLength,
    STAR_EOP_TYPE_EOP);
if (!pTxStreamItem)
{
    (void)puts("Couldn't create the stream item to be transmitted.");
    goto createPacketFailed;
}

/* Create a transfer operation to transmit the packet */
pTxTransOp = STAR_createTxOperation(&pTxStreamItem, 1);
if (!pTxTransOp)
{
    (void)puts("Couldn't create the transmit operation.");
    goto createTxOpFailed;
}

/* Create a transfer operation for receiving one packet */
pRxTransOp = STAR_createRxOperation(1, STAR_RECEIVE_PACKETS);
if (!pRxTransOp)
{
    (void)puts("Couldn't create receive operation, exiting.");
    goto createRxOpFailed;
}

/* Start transmitting the packet */
(void)STAR_submitTransferOperation(channelId, pTxTransOp);

```

```

/* Start receiving a packet */
(void)STAR_submitTransferOperation(channelId, pRxTransOp);

/* Wait indefinitely on a packet being received */
status = STAR_waitOnTransferOperationCompletion(pRxTransOp, -1);
if (status != STAR_TRANSFER_STATUS_COMPLETE)
{
    (void)printf("Could not receive packet, error of %d.\n", status);
    goto rxFailed;
}

/* Check there was one packet received (sanity check, not required) */
packetCount = STAR_getTransferItemCount(pRxTransOp);
if (packetCount != 1)
{
    (void)printf("Unexpected number of packets received: %d.\n", packetCount);
    goto rxFailed;
}

/* Get the stream item received */
pRxStreamItem = STAR_getTransferItem(pRxTransOp, 0);
if (!pRxStreamItem)
{
    (void)puts("Unable to get the stream item received.");
    goto rxFailed;
}

/* Check the stream item is a SpaceWire packet (sanity check, not required) */
if (pRxStreamItem->itemType != STAR_STREAM_ITEM_TYPE_SPACEWIRE_PACKET)
{
    (void)puts("The stream item received is not a packet.");
    goto rxFailed;
}

/* Get the bytes of the packet received */
pReceiveBuffer = STAR_getPacketData(
    (STAR_SPACEWIRE_PACKET *)pRxStreamItem->item, &receiveBufferLength);
if (!pReceiveBuffer)
{
    (void)puts("Unable to get the contents of the packet.");
    goto rxFailed;
}

/* Display the bytes in the packet */
(void)printf("Received packet contents:");
for (i = 0; i < receiveBufferLength; i++)
{
    (void)printf(" %2x", pReceiveBuffer[i]);
}
(void)puts(".");

/* Destroy the buffer containing the received packet */
STAR_destroyPacketData(pReceiveBuffer);

returnStatus = EXIT_SUCCESS;

```

```
rxFailed:
    /* Dispose of the receive operation */
    (void)STAR_disposeTransferOperation(pRxTransOp);

createRxOpFailed:
    /* Dispose of the transmit operation */
    (void)STAR_disposeTransferOperation(pTxTransOp);

createTxOpFailed:
    /* Destroy the transmit packet */
    STAR_destroyStreamItem(pTxStreamItem);

createPacketFailed:
    /* Close the channel to the device */
    (void)STAR_closeChannel(channelId);

    return returnStatus;
}
```

When the above code is run on a PC with a loopback cable connected to port 1 of a PCI Mk2, the following output should be displayed:

```
1 device(s) present.
Device type = SpaceWire PCI Mk2.
Received packet contents: 01 02 03 04 05 06 07 08 09 0a.
```

